UNIVERSIDADE FEDERAL DO CEARÁ

Relatório Final

Disciplina: Qualidade de Software
Equipe: Antonio Felipe Lima Rodrigues (565195) e Lizandra Mara de Souza Pinheiro (566175)
Projeto: PR-Agent, https://github.com/The-PR-Agent/pr-agent
Fork: https://github.com/lizandrapinheiro/pr-agent
Pull Request: https://github.com/The-PR-Agent/pr-agent/pull/4440
Commit SHA: a0dc5515e39775edc4e31bea2175cbea0e9758f1

1. Apresentação do projeto

O PR-Agent é uma ferramenta de código aberto que usa modelos de linguagem para revisar pull requests automaticamente. Ele gera a descrição do PR, uma revisão com pontos de atenção, sugestões de código e respostas a perguntas sobre as mudanças. Funciona com GitHub, GitLab, Bitbucket, Azure DevOps, Gitea, Gerrit e CodeCommit, e é usado por equipes de desenvolvimento que querem acelerar a revisão de código.

Os testes ficam na pasta tests, dividida em três suítes: testes unitários (tests/unittest), testes de ponta a ponta (tests/e2e_tests) e testes de saúde (tests/health_test). As duas últimas exigem tokens e chaves de API. Por isso, seguindo o CONTRIBUTING.md do projeto, usamos apenas a suíte unitária como referência, com cerca de 12.400 casos de teste no commit da atividade.

Os testes são escritos em pytest e usam bastante fixtures, monkeypatch e MagicMock para simular os provedores Git e os modelos de IA.

2. Estado inicial do projeto

2.1 Test smells detectados antes da refatoração (TEMPY)

Test smell	Ocorrências detectadas
Conditional Test Logic	508
Exception Handling	269
Unknown Test	142
Verbose Test	77

Test smell	Ocorrências detectadas
Programming Paradigms Blend	76
Undefined Test	27
Non-Functional Statement	26
Total	1125

2.2 Resultados dos testes e cobertura antes da refatoração (Pytest)

Métrica	Valor
Testes aprovados (Passed)	12.326
Testes com falha (Failed)	43
Erros (Errors)	8
Ignorados (Skipped)	34 (mais 1 xfailed)
Cobertura total (%)	86,66

2.3 Métricas do código de teste antes da refatoração (Radon)

Métrica	Valor
SLOC total dos testes (linha TOTAL com origem = teste)	96.671
Complexidade ciclomática média das funções de teste	2,83 (média de 8.175 funções)
Índice de manutenibilidade médio dos arquivos de teste	45,51 (média de 356 arquivos)

3. Refatorações realizadas

Você deve refatorar 40 ocorrências de 4 tipos diferentes de test smells, priorizando os smells de maior severidade segundo o CATSPY.

3.1 Seleção das ocorrências

Explique quais tipos de test smell você escolheu e por quê, considerando a severidade indicada no CATSPY.

Escolhemos Exception Handling, Verbose Test, Conditional Test Logic e Unknown Test. No CATSPY, Unknown Test, Exception Handling e Conditional Test Logic afetam a confiabilidade do teste, ou seja, podem fazer um teste passar sem verificar o que deveria. Por isso foram tratados como os mais severos. Conditional Test Logic e Exception Handling também eram os mais frequentes no projeto, com 508 e 269 ocorrências. Verbose Test afeta legibilidade, manutenibilidade e eficiência, e completou os quatro tipos.

Outros dois smells que o CATSPY liga à confiabilidade ficaram de fora porque não eram refatoráveis no projeto. As 27 ocorrências de Undefined Test eram funções auxiliares chamadas por outros testes; renomeá-las para começar com test faria o pytest executá-las sozinhas. Quase todos os Non-Functional Statement eram pass obrigatórios em classes vazias usadas como mock.

Dentro de cada tipo, escolhemos ocorrências em que o smell pudesse ser removido por completo sem mudar o que o teste verifica. Em Unknown Test descartamos os falsos positivos do TEMPY, que são testes que verificam exceções com pytest.raises. Em Verbose Test priorizamos os testes mais longos, como o de 111 linhas (R11).

3.2 Lista das refatorações

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R01	Exception Handling	tests/unittest/test_inline_comment_dedup.py::test_iter_unsupported_provider_raises	16aa165	sim	sim	não
R02	Exception Handling	tests/unittest/test_code_suggestions_comment_identity.py::test_custom_suggestions_heading_changes_presentation_only	16aa165	sim	sim	não
R03	Exception Handling	tests/unittest/test_code_suggestions_comment_identity.py::test_suggestions_heading_is_trimmed	16aa165	sim	sim	não
R04	Exception Handling	tests/unittest/test_code_suggestions_comment_identity.py::test_invalid_suggestions_heading_falls_back_to_default	16aa165	sim	sim	não
R05	Exception Handling	tests/unittest/test_code_suggestions_comment_identity.py::test_summarized_artifact_uses_custom_heading_without_identity	16aa165	sim	sim	não
R06	Exception Handling	tests/unittest/test_code_suggestions_comment_identity.py::test_github_check_run_receives_configured_presentation_without_identity	16aa165	sim	sim	não
R07	Exception Handling	tests/unittest/test_webhook_logic_core.py::test_shared_should_process_pr_logic_rules	16aa165	sim	sim	não
R08	Exception Handling	tests/unittest/test_webhook_logic_core.py::test_shared_should_process_pr_logic_payload_parsing	16aa165	sim	sim	não
R09	Exception Handling	tests/unittest/test_webhook_logic_core.py::test_shared_should_process_pr_logic_explicit_empty_overrides_payload	16aa165	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
R10	Exception Handling	tests/unittest/test_webhook_logic_core.py::test_bitbucket_server_should_process_pr_logic_fails_open_on_filter_exception	16aa165	sim	sim	não
R11	Verbose Test	tests/unittest/test_per_directory_settings.py::test_description_questions_and_similar_issue_host_only_keys_are_dropped	6d58b12	sim	sim	não
R12	Verbose Test	tests/unittest/test_codecommit_client.py::test_get_differences	6d58b12	sim	sim	não
R13	Verbose Test	tests/unittest/test_webhook_logic_core.py::test_shared_should_process_pr_logic_rules	6d58b12	sim	sim	não
R14	Verbose Test	tests/unittest/test_get_max_tokens.py::test_claude_model_family_metadata_key_validation	6d58b12	sim	sim	sim: Unknown Test em test_shipped_claude_model_families_pass_validation
R15	Verbose Test	tests/unittest/test_output_models.py::test_required_label_and_list_constraints_are_enforced	6d58b12	sim	sim	não
R16	Verbose Test	tests/unittest/test_codecommit_provider.py::test_add_additional_newlines	6d58b12	sim	sim	não
R17	Verbose Test	tests/unittest/test_get_max_tokens.py::test_claude_opus_4_6_family_expansion_and_capabilities	6d58b12	sim	sim	não
R18	Verbose Test	tests/unittest/test_load_yaml.py::test_load_invalid_yaml1	6d58b12	sim	sim	sim: Unknown Test (falso positivo) em test_invalid_yaml1_is_rejected_by_safe_load
R19	Verbose Test	tests/unittest/test_convert_to_markdown.py::test_can_be_split	6d58b12	sim	sim	não
R20	Verbose Test	tests/unittest/test_bitbucket_provider.py::test_get_diff_files_splits_crlf_headers_after_preamble_in_diffstat_order	6d58b12	sim	sim	não
R21	Conditional Test Logic	tests/unittest/test_effort_bar_clamp.py::test_the_bar_is_always_five_segments	368f467	sim	sim	não
R22	Conditional Test Logic	tests/unittest/test_docker_hardening.py::test_dockerignore_lists_both_supported_secret_files	368f467	sim	sim	não
R23	Conditional Test Logic	tests/unittest/test_diff_pipeline_core.py::test_deletion_only_hunk_is_rendered_when_it_is_the_last_hunk	368f467	sim	sim	não
R24	Conditional Test Logic	tests/unittest/test_diff_pipeline_core.py::test_deletion_only_hunk_is_rendered_when_another_hunk_follows	368f467 (corrigido	sim	sim	não

ID	Test smell	Arquivo::teste	Commit (hash curto)	Testes passaram?	Removido pelo TEMPY?	Introduziu novo smell?
			em 80277fa)			
R25	Conditional Test Logic	tests/unittest/test_extend_patch.py::test_single_hunk	368f467	sim	sim	não
R26	Conditional Test Logic	tests/unittest/test_extend_patch.py::test_single_line_hunk_does_not_resurrect_the_deleted_line	368f467 (corrigido em 80277fa)	sim	sim	não
R27	Conditional Test Logic	tests/unittest/test_extra_config_url.py::test_resolve_rejects_non_string_source	368f467	sim	sim	não
R28	Conditional Test Logic	tests/unittest/test_command_descriptions.py::test_cli_and_bot_help_use_canonical_command_descriptions	368f467	sim	sim	não
R29	Conditional Test Logic	tests/unittest/test_file_filter.py::test_every_supported_platform_actually_drops_a_matching_file	368f467	sim	sim	não
R30	Conditional Test Logic	tests/unittest/test_file_filter.py::test_supported_platforms_do_not_warn	368f467	sim	sim	não
R31	Unknown Test	tests/unittest/test_secret_provider_factory.py::test_accepts_supported_providers	28ceba4	sim	sim	não
R32	Unknown Test	tests/unittest/test_secret_provider_factory.py::test_accepts_unset_provider	28ceba4	sim	sim	não
R33	Unknown Test	tests/unittest/test_mosaico_metadata.py::test_never_raises	28ceba4	sim	sim	não
R34	Unknown Test	tests/unittest/test_mosaico_metadata.py::test_none_meta_is_noop	28ceba4	sim	sim	não
R35	Unknown Test	tests/unittest/test_mosaico_metadata.py::test_log_context_partial_and_empty	28ceba4	sim	sim	não
R36	Unknown Test	tests/unittest/test_mosaico_env_bridge.py::test_empty_meta_is_clean_passthrough	28ceba4	sim	sim	não
R37	Unknown Test	tests/unittest/test_custom_merge_loader_security.py::test_safe_pr_agent_config_does_not_raise	28ceba4	sim	sim	não
R38	Unknown Test	tests/unittest/test_config_loader_secrets.py::test_apply_secrets_manager_config_no_provider	28ceba4	sim	sim	não
R39	Unknown Test	tests/unittest/test_config_loader_secrets.py::test_apply_secrets_manager_config_exception_handling	28ceba4	sim	sim	não
R40	Unknown Test	tests/unittest/test_push_outputs.py::test_errors_are_non_fatal	28ceba4	sim	sim	não

3.3 Exemplos detalhados

R01 – Exception Handling
(tests/unittest/test_inline_comment_dedup.py::test_iter_unsupported_provider_raises)

Antes:

```
def test_iter_unsupported_provider_raises():
    class FooProvider:
        pass
    try:
        list(d.iter_existing_inline_comment_bodies(FooProvider()))
        raise AssertionError("expected NotImplementedError")
    except NotImplementedError:
        pass
```

Depois:

```
def test_iter_unsupported_provider_raises():
    class FooProvider:
        pass
    with pytest.raises(NotImplementedError):
        list(d.iter_existing_inline_comment_bodies(FooProvider()))
```

Por que era um test smell: O teste usava try/except com um raise AssertionError manual para verificar se a função lançava NotImplementedError. A intenção do teste fica escondida no fluxo de controle e o except terminava em pass.

Estratégia do CATSPY aplicada: Substituímos o bloco pelo recurso do framework, como no exemplo do CATSPY. O catálogo usa assertRaises do unittest; no pytest o equivalente é with pytest.raises(NotImplementedError). Não foi preciso outra adaptação.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	2	1
SLOC do arquivo de teste (raw_por_arquivo_e_total)	696	693

R12 – Verbose Test (tests/unittest/test_codecommit_client.py::test_get_differences)

Antes:

```
def test_get_differences(self):
    api = CodeCommitClient()
    api.boto_client = MagicMock()
    api.boto_client.get_paginator.return_value.paginate.return_value = [
        { "differences": [ ... ] }   # 44 linhas com 4 diferenças (M, A, D, R)
    ]

    diffs = api.get_differences("my_test_repo", "commit1", "commit2")

    assert len(diffs) == 4
    assert diffs[0].before_blob_path == "file1.py"
    ...                              # 5 asserts para o arquivo modificado
    assert diffs[1].change_type == "A"
    ...                              # 5 asserts para o arquivo adicionado
    assert diffs[2].change_type == "D"
    ...                              # 5 asserts para o arquivo removido
    assert diffs[3].change_type == "R"
    ...                              # 5 asserts para o arquivo renomeado
```

Depois:

```
_DIFFERENCES_RESPONSE = [ ... ]   # mesma resposta simulada do teste original (44 linhas)

@pytest.fixture
def differences(self):
    # Create a mock CodeCommitClient instance and codecommit_client member
    api = CodeCommitClient()
    api.boto_client = MagicMock()

    # Mock the response from the AWS client for get_differences method
    api.boto_client.get_paginator.return_value.paginate.return_value = self._DIFFERENCES_RESPONSE

    return api.get_differences("my_test_repo", "commit1", "commit2")

def test_get_differences_returns_every_difference(self, differences):
    assert len(differences) == 4

def test_get_differences_maps_modified_file(self, differences):
    assert differences[0].before_blob_path == "file1.py"
    assert differences[0].before_blob_id == "291b15c3ab4219e43a5f4f9091e5a97ee9d7400b"
    assert differences[0].after_blob_path == "file1.py"
    assert differences[0].after_blob_id == "46ad86582da03cc34c804c24b17976571bca1eba"
    assert differences[0].change_type == "M"

def test_get_differences_maps_added_file(self, differences):
    assert differences[1].before_blob_path == ""
    assert differences[1].before_blob_id == ""
    assert differences[1].after_blob_path == "file2.py"
    assert differences[1].after_blob_id == "2404c7874fcbd684d6779c1420072f088647fd79"
    assert differences[1].change_type == "A"

def test_get_differences_maps_deleted_file(self, differences):
    assert differences[2].before_blob_path == "file3.py"
    assert differences[2].before_blob_id == "9af7989045ce40e9478ebb8089dfbadac19a9cde"
    assert differences[2].after_blob_path == ""
    assert differences[2].after_blob_id == ""
    assert differences[2].change_type == "D"

def test_get_differences_maps_renamed_file(self, differences):
    assert differences[3].before_blob_path == "file5.py"
    assert differences[3].before_blob_id == "738e36eec120ef9d6393a149252698f49156d5b4"
    assert differences[3].after_blob_path == "file6.py"
    assert differences[3].after_blob_id == "faecdb85f7ba199df927a783b261378a1baeca85"
    assert differences[3].change_type == "R"
```

Por que era um test smell: Com 74 linhas e 21 asserts, o teste verificava quatro tipos de mudança (modificado, adicionado, removido e renomeado) de uma vez. Uma falha não indicava qual tipo estava errado.

Estratégia do CATSPY aplicada: Seguimos o CATSPY e dividimos em testes menores, um por responsabilidade: a quantidade de diferenças e um teste para cada tipo de mudança. A resposta simulada da AWS virou o atributo de classe _DIFFERENCES_RESPONSE e a chamada virou a fixture differences. A adaptação foi não colocar os dados dentro da fixture, que ficaria longa e também seria apontada pelo TEMPY.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	22	6 (maior dos 5

Métrica	Antes	Depois
		testes novos)
SLOC do arquivo de teste (raw_por_arquivo_e_total)	210	218

R21 – Conditional Test Logic
(tests/unittest/test_effort_bar_clamp.py::test_the_bar_is_always_five_segments)

Antes:

```
def test_the_bar_is_always_five_segments():
    """Keep the total segment count constant so the bar reads as a 1-5 scale."""
    for value in ("0", "1", "3", "5", "8", "99"):
        blue, white = _bars(value)
        assert blue + white == 5, value
```

Depois:

```
@pytest.mark.parametrize("value", ["0", "1", "3", "5", "8", "99"])
def test_the_bar_is_always_five_segments(value):
    """Keep the total segment count constant so the bar reads as a 1-5 scale."""
    blue, white = _bars(value)
    assert blue + white == 5, value
```

Por que era um test smell: O teste percorria seis valores com um for. Se um valor falhasse, os seguintes não eram testados, e o relatório não mostrava qual caso quebrou.

Estratégia do CATSPY aplicada: O CATSPY propõe extrair cada caso do condicional para um teste próprio. Usamos @pytest.mark.parametrize, que cria uma execução separada para cada valor sem duplicar código.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	3	2
SLOC do arquivo de teste (raw_por_arquivo_e_total)	40	40

R34 – Unknown Test (tests/unittest/test_mosaico_metadata.py::test_none_meta_is_noop)

Antes:

```
def test_none_meta_is_noop(self):
    with langfuse_span(None, "ctx"):
        pass  # must not raise
```

Depois:

```
def test_none_meta_is_noop(self):
    ran = {"v": False}
    with langfuse_span(None, "ctx"):
        ran["v"] = True
    assert ran["v"] is True
```

Por que era um test smell: O teste só abria um bloco de contexto e executava pass, sem nenhuma asserção. Ele passava sempre que nada lançasse exceção, sem deixar claro o resultado esperado. O pass também era apontado como Non-Functional Statement.

Estratégia do CATSPY aplicada: Como no CATSPY, adicionamos uma asserção explícita. Um indicador registra que o bloco foi executado, e a asserção final confirma isso, tornando visível o que antes só era verificado de forma implícita. Como efeito colateral, o pass saiu e o Non-Functional Statement também foi removido.

Métricas do teste refatorado:

Métrica	Antes	Depois
Complexidade ciclomática da função (cc_por_funcao)	1	2
SLOC do arquivo de teste (raw_por_arquivo_e_total)	103	107

4. Estado final e comparação

Métrica	Antes	Depois	Variação
Total de test smells (TEMPY)	1.125	1.079	-46
Testes aprovados	12.326	12.399	+73
Testes com falha	43	43	0
Cobertura total (%)	86,66	86,67	+0,01 p.p.
SLOC dos testes	96.671	96.687	+16

Métrica	Antes	Depois	Variação
Complexidade ciclomática média das funções de teste	2,825	2,818	-0,007
Índice de manutenibilidade médio dos arquivos de teste	45,51	45,48	-0,03

Test smell	Antes	Depois	Variação
Conditional Test Logic	508	496	-12
Exception Handling	269	259	-10
Unknown Test	142	134	-8
Verbose Test	77	65	-12
Programming Paradigms Blend	76	76	0
Undefined Test	27	27	0
Non-Functional Statement	26	22	-4
Total	1125	1079	-46

5. Análise dos resultados

Quais métricas melhoraram e quais pioraram?

Melhoraram o total de test smells, que caiu de 1.125 para 1.079 (redução de 46), e o número de testes aprovados, que subiu de 12.326 para 12.399, porque as divisões e parametrizações criaram 73 casos de teste novos. A complexidade ciclomática média das funções de teste também caiu, de 2,825 para 2,818, já que os loops e condicionais saíram dos testes.

Pioraram levemente o SLOC dos testes, que subiu de 96.671 para 96.687 (16 linhas a mais, por causa das fixtures, decorators e testes novos), e o índice de manutenibilidade médio dos arquivos de teste, que caiu de 45,51 para 45,48. Essa variação é muito pequena e vem dos arquivos que ganharam funções e atributos de classe.

O código-fonte do projeto não foi alterado.

Alguma refatoração introduziu novos test smells? Quais?

Sim. Duas refatorações de Verbose Test fizeram o TEMPY apontar dois Unknown Test em testes novos. Eles foram identificados nas conferências feitas após cada refatoração, e optamos por não refatorá-los pelos motivos abaixo.

1. O enunciado da atividade permite: smells novos de outros tipos não precisam ser refatorados, pois também são dados da pesquisa.

2. Não houve perda de verificação. Na R14, o trecho que valida as famílias de modelo já não tinha asserção no teste original, e o teste verifica que a validação não lança exceção. A divisão apenas tornou visível um Unknown Test que já existia dentro do teste longo. Na R18, o teste novo verifica a exceção com pytest.raises(ScannerError), que o TEMPY não reconhece como asserção, um falso positivo da ferramenta.

3. Uma correção seria artificial. Para o TEMPY deixar de apontar esses testes, seria necessário acrescentar asserções sem utilidade, como verificar que uma função que sempre retorna None retornou None. Isso adicionaria código sem relação com um problema real. No próprio PR, o CodeQL apontou esse tipo de asserção em outro teste como uso indevido do retorno de um procedimento.

Além disso, dois Conditional Test Logic aparecem em métodos com nomes novos (R14 e R17), mas são os mesmos for que já existiam nos testes originais. Eles apenas passaram para os testes menores, e a quantidade de Conditional Test Logic no arquivo não mudou.

O resultado dos testes ou a cobertura mudou?

Os testes que passavam antes continuaram passando. Os aprovados foram de 12.326 para 12.399, e as falhas ficaram em 43 nos dois momentos. Todas são ligadas ao Windows: os.getuid, criação de symlinks sem privilégio, quebras de linha CRLF e limite de tamanho de variável de ambiente.

Durante a refatoração, um arquivo de testes que não alteramos teve falhas intermitentes. Os testes de cancelamento desse arquivo comparam tempos com limite de 0,1 segundo, e o relógio do Windows produz valores como 0,10000000000582077. Com git stash confirmamos que ele falha da mesma forma sem as refatorações.

A cobertura passou de 86,66% para 86,67%, praticamente igual, o que era esperado porque só o código de teste foi alterado.

Quais tipos de smell foram mais fáceis e mais difíceis de refatorar?

Os mais fáceis foram Unknown Test e Conditional Test Logic com for sobre valores fixos: bastou acrescentar a asserção ou trocar o for por @pytest.mark.parametrize.

Exception Handling teve dificuldade média. O caso com try/except virou pytest.raises diretamente, mas a maioria dos casos era try/finally de restauração de configuração, que exigiu criar fixtures com yield.

Verbose Test foi o mais difícil. Foi preciso entender cada teste para dividi-lo por responsabilidade sem mudar o que ele verifica, tratar a dependência de ordem entre os blocos (R13) e evitar que os dados grandes fossem para fixtures longas, que o TEMPY também conta como Verbose Test.

6. Uso do CATSPY

Como você usou o CATSPY durante a atividade?

Consultamos a página de cada smell antes de escolher as ocorrências e durante cada refatoração.

Usamos a definição e o critério de detecção para confirmar se a ocorrência apontada pelo TEMPY era o smell de fato, as categorias de impacto (legibilidade, manutenibilidade e confiabilidade) para priorizar os tipos, e o exemplo refatorado como referência da técnica a aplicar. Os links das páginas consultadas estão no diário de bordo.

Quais partes do catálogo mais ajudaram (definição, efeitos negativos, exemplo com problema, exemplo refatorado)?

O exemplo refatorado e a definição foram as partes mais úteis. O exemplo deixou claro qual técnica usar: assertRaises para Exception Handling, dividir o teste para Verbose Test, extrair os casos para Conditional Test Logic e adicionar asserção para Unknown Test.

As categorias de impacto ajudaram a justificar a prioridade dos smells.

Houve casos em que a estratégia do catálogo não se aplicava diretamente ao projeto? Como você resolveu?

Sim. Em Exception Handling, o catálogo mostra apenas try/except com assertRaises, mas 9 das 10 ocorrências eram try/finally usados só para restaurar configurações globais. Com autorização do monitor, adaptamos a refatoração movendo a restauração para fixtures do pytest com yield.

Em Verbose Test, dois testes (R19 e R20) tinham uma única responsabilidade e eram longos só por causa dos dados. Em vez de dividir, extraímos os dados para atributos de classe.

Os exemplos do catálogo usam unittest, então traduzimos as técnicas para os equivalentes do pytest: pytest.raises, fixtures e parametrize.

O que faltou ou poderia ser melhorado no catálogo?

Faltaram exemplos em pytest, já que o projeto e muitos projetos atuais não usam unittest, e exemplos de Exception Handling com try/finally de limpeza.

Também ajudaria uma seção sobre falsos positivos das ferramentas, como testes com pytest.raises apontados como Unknown Test, e sobre como tratar testes cujo objetivo é apenas não lançar exceção. Nesse ponto, uma asserção adicionada para remover um Unknown Test foi apontada pelo CodeQL no PR como uso do retorno de um procedimento que sempre devolve None.

Por fim, o catálogo fala em 30 linhas para Verbose Test, enquanto o TEMPY usa 45, e valeria explicar essa diferença.

7. Divisão do trabalho

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
Lizandra Mara de Souza Pinheiro	R01 a R20 (Exception Handling e Verbose Test)	20

Integrante	Refatorações realizadas (IDs da Seção 3.2)	Quantidade
Antonio Felipe Lima Rodrigues	R21 a R40 (Conditional Test Logic e Unknown Test)	20

8. Dificuldades e lições aprendidas

Quais foram as principais dificuldades (ambiente, dependências, entendimento do código, ferramentas, revisão do PR)?

Ambiente: o projeto foi feito para Linux e rodamos no Windows. Foi preciso instalar o Python 3.12, usar o uv como manda o CONTRIBUTING.md, definir as variáveis PYTHONPATH e PYTHONUTF8 e conviver com 43 testes que falham só no Windows. A pasta do projeto no OneDrive também causou travamentos de arquivos no ambiente virtual.

Ferramentas: o TEMPY aponta falsos positivos (Unknown Test em testes com pytest.raises e Undefined Test em funções auxiliares) e considera qualquer bloco try como Exception Handling, inclusive try/finally. O pre-commit acusou quebras de linha misturadas que já existiam no projeto; não normalizamos para não alterar arquivos inteiros.

Revisão do PR: a revisão automática e um mantenedor apontaram duas parametrizações que enfraqueceram asserções, uma por um espaço inicial removido e outra por um valor omitido. Elas foram corrigidas no commit 80277fa. Também foi preciso resolver conflitos com a versão mais nova do projeto, incluindo uma mudança no valor esperado em um dos testes refatorados.

O que você aprendeu com a atividade?

Que remover um smell apontado por uma ferramenta não basta: cada mudança precisa preservar exatamente o que o teste verificava, e uma parametrização mal feita pode enfraquecer o teste sem que nenhuma falha apareça.

Aprendemos a usar fixtures, parametrize e pytest.raises, a medir o efeito das mudanças com Radon, pytest-cov e TEMPY, a interpretar os falsos positivos das ferramentas e a contribuir com um projeto real, seguindo suas regras de commit, lint e revisão.

9. Contribuição (Pull Request)

O PR #4440, com o título refactor(tests): remove test smells from unit tests, foi aberto para o repositório original com 24 arquivos de teste, um commit por tipo de smell e uma descrição em inglês explicando as refatorações, como foram testadas e as adaptações feitas.

A revisão automática do próprio PR-Agent e o mantenedor IsmaelMartinez apontaram que duas parametrizações de Conditional Test Logic (R24 e R26) deixavam passar um bug: uma perdeu o espaço inicial que identifica linhas de contexto e a outra omitiu um dos valores verificados. Corrigimos no commit 80277fa.

O mantenedor também pediu a atualização com a versão mais nova do projeto. Fizemos o merge (commit b8dd32c), resolvendo conflitos em três arquivos e atualizando um teste refatorado para o novo valor esperado da janela de contexto de um modelo.

O mantenedor aprovou o PR, agradecendo as correções e a resolução do merge, e o PR foi mesclado ao projeto em 28/10/2026. O CodeQL deixou dois avisos sobre asserções adicionadas em testes de Unknown Test (R31 e R32), que verificam o retorno de uma função que sempre devolve None.

10. Observações gerais

As medições oficiais foram feitas no Windows 11 com Python 3.12, com as variáveis PYTHONPATH e PYTHONUTF8 definidas nas duas etapas, e a suíte de referência foi a de testes unitários.

As métricas depois foram medidas no commit 28ceba43, antes da correção 80277fa sugerida na revisão do PR. Essa correção não altera a contagem de smells.

As refatorações de Exception Handling também removeram, como efeito colateral, 2 Conditional Test Logic, 1 Non-Functional Statement e 2 Verbose Test. As de Unknown Test removeram 3 Non-Functional Statement.

As pastas de métricas e os scripts estão na branch metricas do fork: https://github.com/lizandrapinheiro/pr-agent/tree/metricas

Sugestão para o TEMPY: reconhecer pytest.raises como asserção e não tratar try/finally como Exception Handling.

O passo a passo da preparação do ambiente, das medições e do fluxo de cada refatoração está no Apêndice A.

Apêndice A. Ambiente e procedimento

A.1 Ambiente

Windows 11, Python 3.12 e o gerenciador de pacotes uv, indicado no CONTRIBUTING.md do projeto. A pasta do projeto ficou no OneDrive, o que causou alguns travamentos de arquivos no ambiente virtual. Usamos como referência apenas a suíte de testes unitários (tests/unittest), porque as suítes de ponta a ponta e de saúde exigem tokens e chaves de API.

Não foi usado arquivo .env. A configuração foi feita pelo ambiente virtual criado pelo uv e por duas variáveis de ambiente definidas em cada terminal:

```
$env:PYTHONPATH="."
$env:PYTHONUTF8="1"
```

PYTHONPATH é exigida pelo projeto para que os testes encontrem o pacote pr_agent. PYTHONUTF8 faz o Python ler os arquivos em UTF-8 no Windows; sem ela, testes que leem arquivos com caracteres especiais falhavam com UnicodeDecodeError. As duas foram usadas nas medições de antes e de depois e durante toda a refatoração.

A.2 Preparação do projeto

Fizemos o fork do repositório no GitHub, clonamos o fork e criamos a branch de trabalho a partir do commit indicado na planilha. Em seguida, instalamos as dependências com o uv e ativamos o ambiente virtual:

```
git clone https://github.com/lizandrapinheiro/pr-agent.git
git checkout -b refatoracao-test-smells a0dc5515e39775edc4e31bea2175cbea0e9758f1
uv sync
.venv\Scripts\Activate.ps1
```

O TEMPY foi clonado na pasta acima do projeto, fora do repositório. As ferramentas de medição foram instaladas no mesmo ambiente virtual:

```
git clone https://github.com/danieldavidf/TEMPY.git ..\TEMPY
uv pip install radon pytest-html pytest-cov
```

Para que os scripts de extração e as pastas de métricas não entrassem nos commits nem no PR, eles foram adicionados ao arquivo .git/info/exclude.

A.3 Medições de antes e de depois

Cada script foi executado uma única vez antes de qualquer alteração (etapa before) e uma única vez depois de todas as refatorações (etapa after), sempre a partir da raiz do projeto e com as mesmas variáveis de ambiente:

```
python extract_metrics_radon.py --etapa before
python extract_smells_tempy.py --etapa before
python extract_metrics_pytest.py --etapa before
```

Nos scripts do Radon e do pytest, o código-fonte foi configurado como pr_agent, e no do pytest a pasta de testes foi configurada como tests/unittest. As pastas geradas (metrics-before-* e metrics-after-*) estão na branch metricas do fork. As medições de depois foram feitas no commit 28ceba43.

A.4 Fluxo de cada tipo de smell

1. Criar uma branch a partir da branch principal de trabalho, por exemplo refactor/remove-test-smell-exception-handling.

2. Consultar a página do smell no CATSPY e conferir no código se a ocorrência apontada pelo TEMPY era o smell de fato.

3. Editar apenas o necessário para remover o smell, sem mudar o que o teste verifica.

4. Rodar os testes do arquivo alterado e conferir o arquivo no TEMPY, comparando com o smells.csv de antes para garantir que a ocorrência sumiu e que nenhum smell novo apareceu.

5. Rodar o ruff no arquivo alterado.

6. Ao terminar as 10 ocorrências do tipo, rodar a suíte completa e reexecutar as falhas para separar as falhas do Windows e as intermitentes de problemas reais.

7. Fazer um commit com todas as ocorrências do tipo, no padrão refactor: remove <smell> em <arquivos>, enviar para o fork e fazer o merge na branch principal de trabalho.

8. Registrar no diário de bordo as ocorrências, as dificuldades, o uso do CATSPY e a verificação.

A.5 Comandos usados durante a refatoração

Testes de um arquivo e da suíte completa:

```
uv run pytest tests/unittest/<arquivo>.py -q -p no:warnings
uv run pytest tests/unittest -q -p no:warnings
```

Reexecução apenas das falhas da última execução:

```
uv run pytest tests/unittest -q -p no:warnings --lf
```

Conferência do TEMPY sem salvar resultados, em um arquivo e no projeto todo:

```
python extract_smells_tempy.py --arquivo tests/unittest/<arquivo>.py
python extract_smells_tempy.py
```

Smells do arquivo na medição de antes, para comparação:

```
Select-String -Path metrics-before-tempy\smells.csv -Pattern "<arquivo>"
```

Lint e verificações do projeto:

```
uv run ruff check tests/unittest/<arquivo>.py
uv run pre-commit run --files <arquivos alterados>
```